OpenMP — Lesson 4

Sharing the work: the `for` construct

Gustavo Lozano

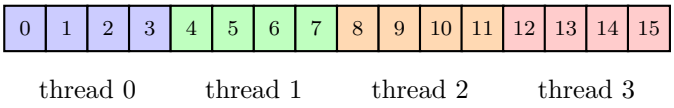
November 2026

Source: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (the loop work-sharing construct and the combined `parallel for`).

1 The idea

In Lessons 1–3 every thread executed the *whole* block, so with 4 threads the loop ran 4 times. That shows how threads behave, but it does not make a program faster. To gain speed, the threads must **divide** the work: each iteration of the loop is executed by exactly *one* thread. This is a **work-sharing** construct, and the most important one is `for`.

16 iterations, 4 threads: each gets one contiguous block of 4



2 The syntax

```
#pragma omp parallel for          /* create a team AND split the next loop */
for (int i = 0; i < n; i++)
    work(i);
```

- `parallel for` is a shortcut for a `parallel` region containing only an `omp for` loop.
- The loop index `i` is **private** automatically.
- There is an **implicit barrier** at the end of the loop: after it, all iterations are done.
- The loop must have a countable form (e.g. `for (i = a; i < b; i++)`), so OpenMP can compute the number of iterations before it starts.
- **You** must guarantee the iterations are **independent**. OpenMP does not check. Shared results are combined with `reduction` (Lesson 3).

How iterations are assigned to threads is called the *schedule*. With the default in GCC, each thread receives one contiguous block of about n/p iterations. Other schedules are the topic of Lesson 5.

3 The example: computing π

We use the identity

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx \;\approx\; h\sum_{i=0}^{n-1}\frac{4}{1+x_i^2}, \qquad x_i = (i+\tfrac{1}{2})\,h, \quad h = \frac{1}{n},$$

(the midpoint rule). Each term depends only on i, so the iterations are independent, and the only shared result, `sum`, is combined with `reduction(+:sum)`.

4 The code

parallel_for.c

```c
/*
 * parallel_for.c -- OpenMP Lesson 4: sharing the work of a loop.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (the loop work-sharing construct, combined parallel for).
 *
 * Compile:
 *     gcc -Wall -fopenmp -o parallel_for parallel_for.c
 * Run:
 *     OMP_NUM_THREADS=4 ./parallel_for
 */

#include <stdio.h>
#include <omp.h>

#define SHOW  16           /* small loop: to SEE who does what         */
#define STEPS 100000000    /* big loop: to MEASURE the speed-up        */

int main(void)
{
    /* ---- Part 1: who does which iteration? ----
       "parallel for" = create a team AND split the loop among it.
       Each iteration is done by exactly ONE thread (not by all, as in
       Lessons 2-3). The loop index i is private automatically.         */
    #pragma omp parallel for
    for (int i = 0; i < SHOW; i++)
        printf("  iteration %2d -> thread %d\n", i, omp_get_thread_num());
    /* implicit barrier here: all iterations are finished after the loop */

    /* ---- Part 2: a loop worth parallelising ----
       pi = integral from 0 to 1 of 4/(1+x^2) dx, by the midpoint rule.
       Iterations are independent; only sum is combined -> reduction.   */
    double step = 1.0 / STEPS;
    double sum  = 0.0;

    double t = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for (long i = 0; i < STEPS; i++) {
        double x = (i + 0.5) * step;      /* declared inside -> private */
        sum += 4.0 / (1.0 + x * x);
    }
    t = omp_get_wtime() - t;

    printf("\nthreads = %d   pi = %.12f   time = %.4f s\n",
           omp_get_max_threads(), step * sum, t);
    return 0;
}
```

```
gcc -Wall -fopenmp -o parallel_for parallel_for.c
OMP_NUM_THREADS=4 ./parallel_for
```

Typical output (order and times will differ):

```
  iteration  4 -> thread 1
  iteration  5 -> thread 1
  ...
  iteration  0 -> thread 0
  ...
  iteration 15 -> thread 3

threads = 4   pi = 3.141592653590   time = 0.1064 s
```

Key ideas

- Each iteration runs exactly once, on one thread: the work is *divided*, not repeated.
- By default thread k gets the k-th contiguous block of iterations.
- The printing order is still not deterministic, but the *assignment* (which thread does which iteration) is fixed by the schedule.
- The value of π is the same with any number of threads (up to rounding in the last digits, because floating-point addition is not associative and the reduction adds in a different order).

5 The experiment

Experiment

Question: how much faster does the loop get as threads are added?

1. Check your number of cores: `nproc`.
2. Run the program with several thread counts. Keep only the last line:

```
for t in 1 2 4 8; do OMP_NUM_THREADS=$t ./parallel_for | tail -1; done
```

3. For each p compute the **speed-up** and the **efficiency**:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}.$$

What to observe and answer.

1. In the ideal case $S(p) = p$. How close do you get while $p \le$ `nproc`?
2. What happens to $S(p)$ when $p >$ `nproc`? Why?
3. Why is $S(p)$ a bit below p even when there are enough cores?

Answers (read after trying). (1) This loop is almost perfectly parallel: the iterations are independent and need no synchronization. Expect $S(p)$ close to p; for example, on a 2-core machine $T(1) = 0.20\,\text{s}$ and $T(2) = 0.12\,\text{s}$ give $S(2) \approx 1.7$. (2) It stops growing. Once every core is busy, extra threads only take turns on the same cores and add switching overhead. Threads

are not cores (Lesson 1). (3) Creating the team, splitting the iterations, the barrier and the final reduction all cost time that the serial program does not pay. This *parallel overhead* is small here, but it is never zero.

Next lesson. The `schedule` clause: `static`, `dynamic` and `guided`, and why the default split fails when iterations do unequal amounts of work (Chapman et al., Ch. 4).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.